PHP Connecting and Querying

Forms

Definition

An **HTML form** is a section of a webpage used to collect input from users. This input is typically sent to a server for processing, such as saving data to a database, authenticating users, or performing searches.

Forms are created using the `<form>` element and contain interactive controls like text fields, checkboxes, radio buttons, and buttons.

Basic Syntax

```
<form action="process.php" method="POST">
    <input type="text" name="username" required>
    <input type="submit" value="Submit">
</form>
```

1. The **<form>** Element

The <form> tag defines the form container and controls how data is sent.

*a. **action** Attribute*

The **action** specifies where the form data will be sent after submission.

- It can be:
  - A PHP file (process.php)
  - A URL (https://example.com/submit)
- If omitted, the form submits to the same page.

*b. **method** Attribute*

The **method** defines how the data is sent to the server.

GET	POST
• Data is appended to the URL • Visible in URL • Limited data size • Less secure • Can be bookmarked	• Data is sent in the request body • Not visible in URL • Supports larger data • More secure than GET • Cannot be bookmarked

Database Connection - mysqli_connect()

Definition

`mysqli_connect()` is a built-in PHP function used to **establish a connection to a MySQL database server**.

It is part of the **MySQLi (MySQL Improved)** extension, which provides improved features over the older `mysql_*` functions, such as better security, prepared statements, and support for modern MySQL features.

Syntax

```
$con = mysqli_connect(host, username, password, database, port, socket)
```

Parameters

Parameter	Description
host	The server where MySQL is running
username	MySQL username
password	Password for the MySQL user
database (*optional*)	The database you want to connect to. If omitted, you can select it later using: `mysqli_select_db()`
port (*optional*)	MySQL port number
socket (*optional*)	Used for socket connections (rarely used in basic setups)

Basic Example

```
<?php
    $con = mysqli_connect("localhost", "root", "", "db_pgpc");
?>
```

Executing Queries - mysqli_query()

Definition

`mysqli_query()` is a PHP function used to **execute SQL queries** on a MySQL database after a connection has been established using `mysqli_connect()`.

It allows you to perform operations such as:

- Retrieving data (`SELECT`)
- Inserting data (`INSERT`)
- Updating data (`UPDATE`)
- Deleting data (`DELETE`)

Syntax

```
mysqli_query(connection, query);
```

Parameters

Parameter	Description
connection	The MySQL connection created using `mysqli_connect()`
query	The SQL statement you want to execute

Example

```
<?php
    $con = mysqli_connect("localhost", "root", "", "db_pgpc");
    $query = "INSERT INTO users (fullname, age) VALUES ('Harold Inacay', 25)";
    $result = mysqli_query($con, $query);
?>
```

Error Handling

Definition

Error handling in MySQLi refers to the process of **checking and managing errors that occur during database operations**, such as:

- Connection failures
- Invalid SQL queries
- Data insertion/update issues

It ensures that your application:

- Does not crash unexpectedly
- Provides useful debugging information
- Protects sensitive database details

1. Handling Connection Errors

`mysqli_connect_error()` - returns the error message from the connection attempt

```
<?php
    $con = mysqli_connect("localhost", "root", "", "mydb");
    if (!$con) {
        die("Connection failed: " . mysqli_connect_error());
    }
?>
```

2. Handling Query Errors

`mysqli_error($con)` - returns last query error

```
<?php
    $query = "SELECT * FROM users";
    $result = mysqli_query($con, $query);
    if (!$result) {
        echo "Query failed: " . mysqli_error($con);
    }

    $query = "UPDATE users SET name='Jane' WHERE id=1";
    if (!mysqli_query($con, $query)) {
        echo "Update failed: " . mysqli_error($con);
    }
?>
```

*3. Using **mysqli_errno()***

`mysqli_errno($con)` - returns the numeric error code.

```
<?php
    if (!$result) {
        echo "Error Code: " . mysqli_errno($conn);
        echo "Error Message: " . mysqli_error($conn);
    }
?>
```

*4. Using **or die()** Pattern*

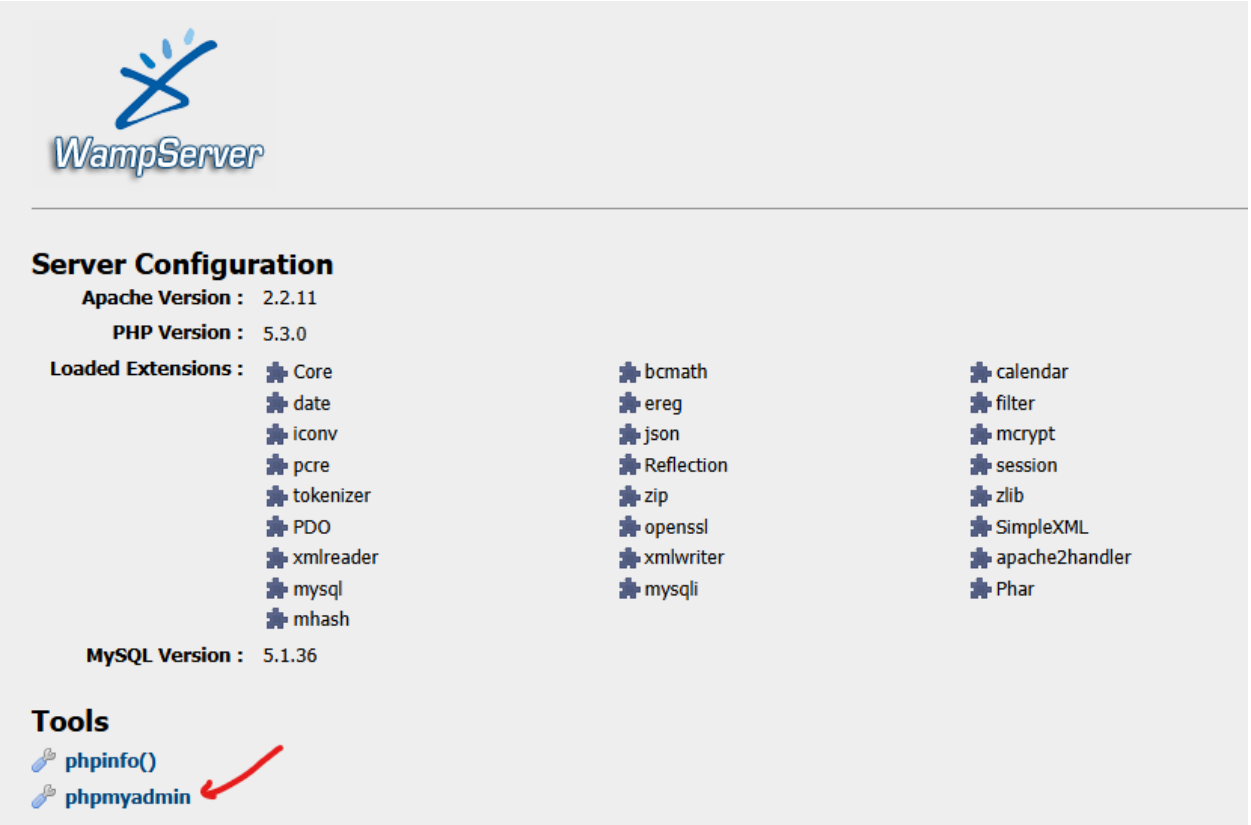
`or die()` – a quick way to stop execution if an error occurs.

```
<?php
    $result = mysqli_query($conn, "SELECT * FROM users") or die("Query failed: "
. mysqli_error($con));
?>
```

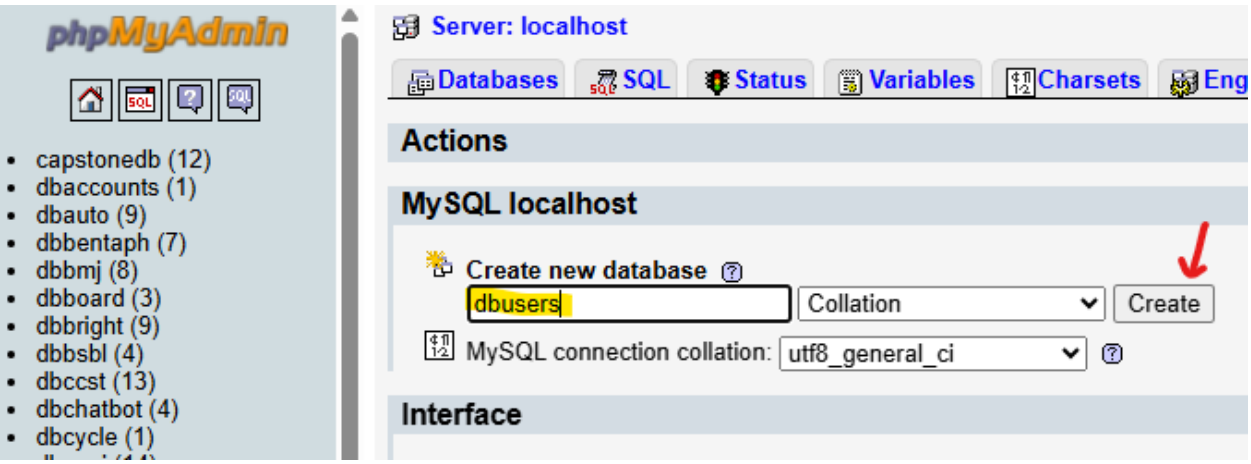
PHP + MYSQL Sample Codes

Create Database

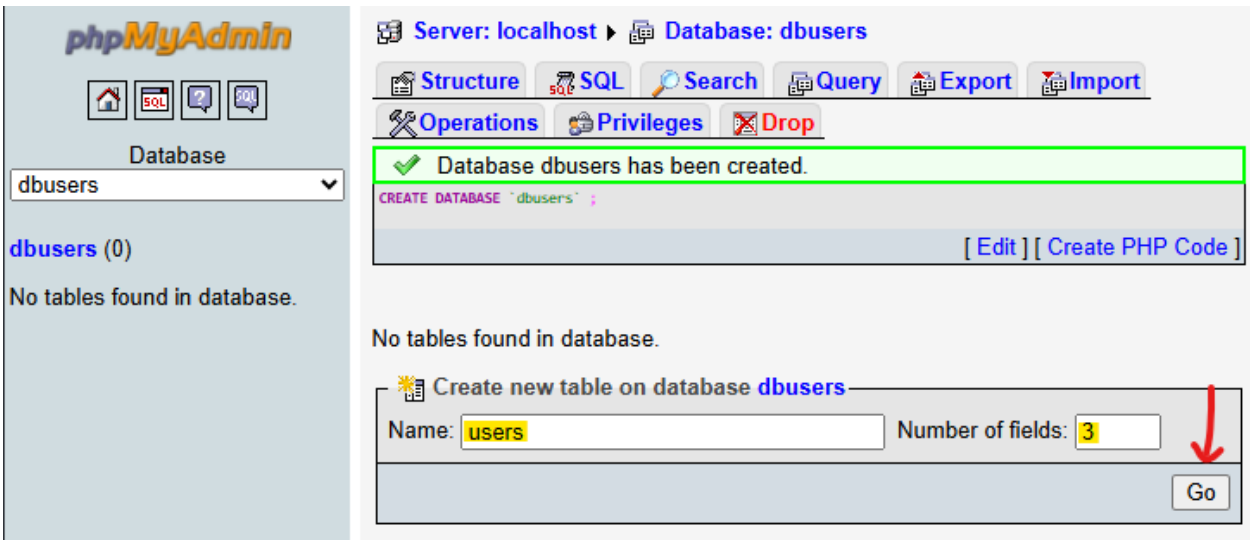
1. Open **WampServer** (access **localhost** in browser).
2. Click **phpMyAdmin**.



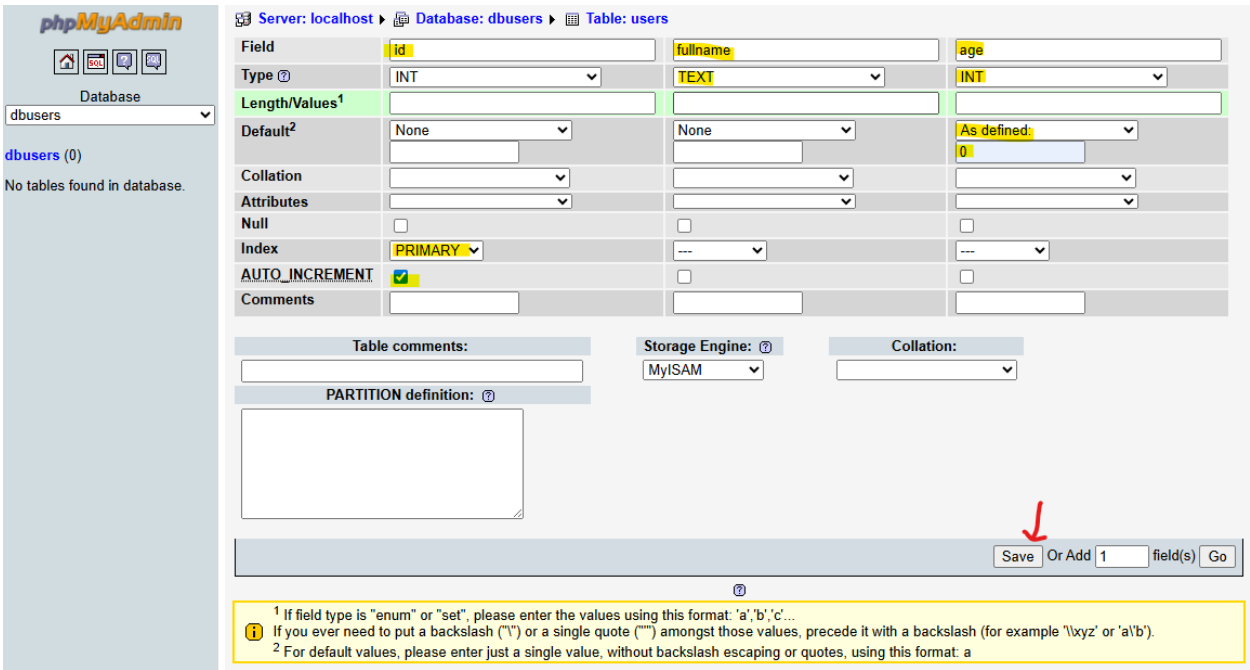
3. Enter **database name** and click **Create.**



4. Enter **table name** and **Number of fields** then click **Go**.

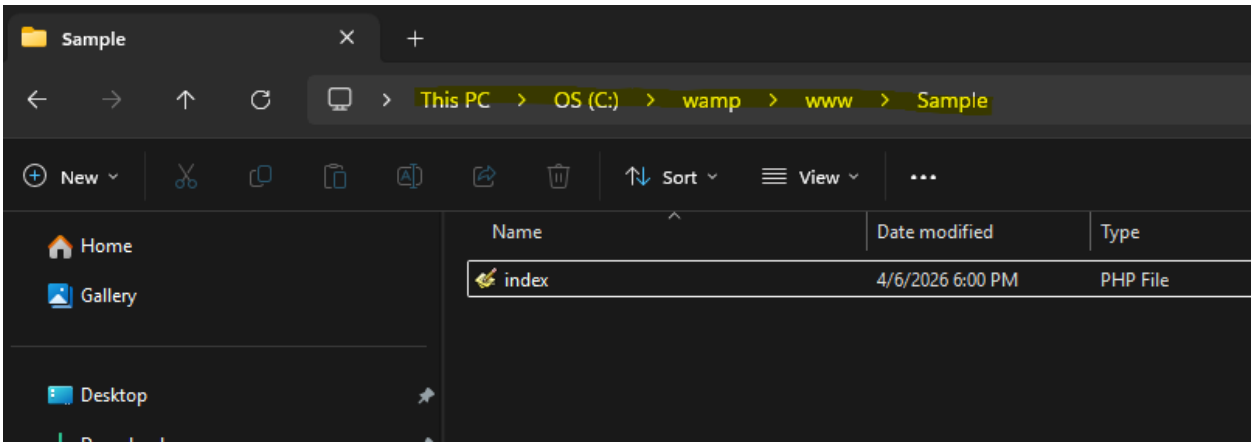


5. Enter the **column names** and the correct **data types** then click **Save**.



Create Website

1. Create **index.php** file in **C:/wamp/www/<YOUR_FOLDER>/**



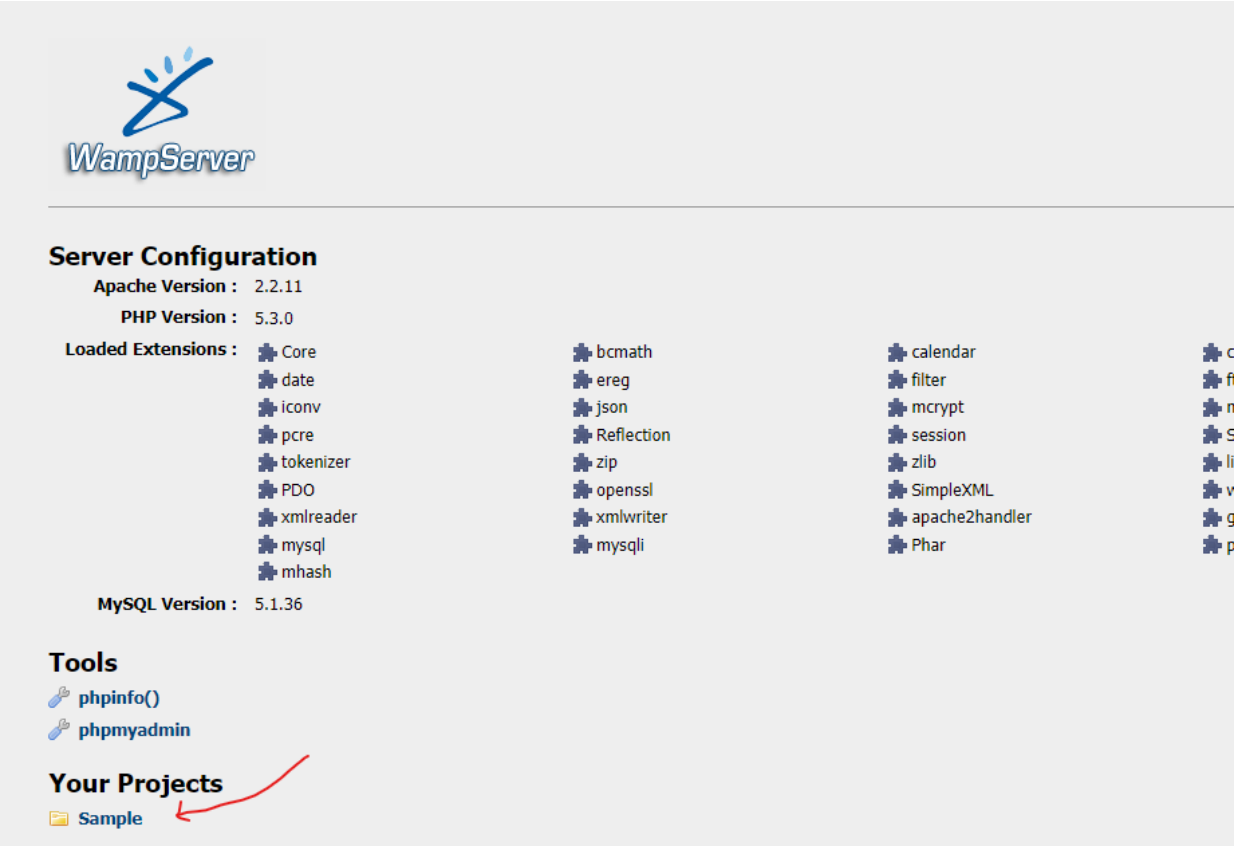
2. Use the following codes for **index.php**:

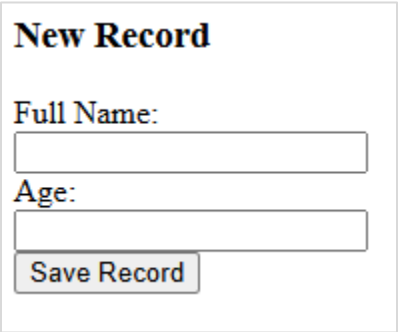
```
<?php
    $con = mysqli_connect("localhost", "root", "", "dbusers");
?>
<html>
    <body>
        <h3>New Record</h3>
        <form method="POST">
            <label>Full Name:</label><br/>
            <input type="text" name="fullname" required><br/>
            <label>Age:</label><br/>
            <input type="number" name="age" required><br/>
            <input type="submit" name="btnsubmit" value="Save Record">
            <?php
                if(isset($_POST["btnsubmit"])){
                    $fullname = $_POST["fullname"];
                    $age = $_POST["age"];
                    mysqli_query($con, "insert into users(fullname, age)
   values('$fullname',$age)");
                }
            ?>
        </form>
    </body>
</html>
```

View Website

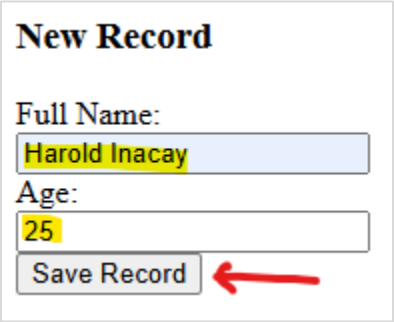
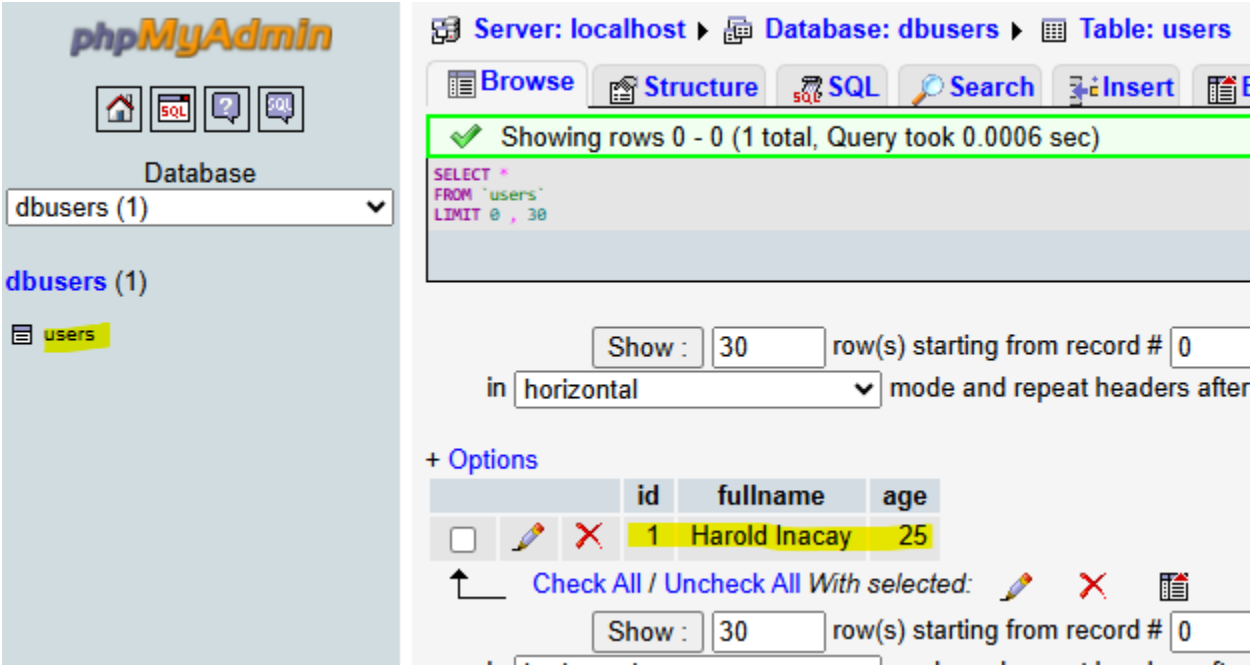
Note: Make sure that you saved your **PHP file** in "**C:/wamp/www/<YOUR_FOLDER>/**". In this sample, I created "**Sample**" folder for my project.

1. Open **WampServer** (access **localhost** in browser).
2. Under **Your Projects** section, select the **Folder** you created.





3. To test, complete the form then click **Save Record** button.



4. Check your database, click the table (**users**), then check if the record has been added.